Programación Orientada a Objetos (POO), Clases y Objetos en Java con IntelliJ IDEA

Taller Práctico Intensivo:

1. Introducción y Guía de Entorno de Trabajo (IntelliJ IDEA)

Bienvenido al taller práctico de Programación Orientada a Objetos (POO) en Java. El propósito de esta guía es consolidar de manera autónoma los conceptos fundamentales de diseño de clases, instanciación de objetos, encapsulamiento básico, métodos de instancia y estáticos, sobrecarga de constructores y depuración de código en el entorno de desarrollo IntelliJ IDEA.

Para la resolución exitosa de este taller, se requiere integrar de forma transversal los contenidos del Módulo 1: tipos de datos primitivos y Wrappers, condicionales (`if-else` / `switch`), bucles (`for` / `while` / `do-while`), manipulación de cadenas (`String` / `StringBuilder`) y salida formateada en consola mediante `System.out.printf` .

Reglas del Taller y Configuración Inicial en IntelliJ IDEA

- **Creación del Proyecto:** Inicie IntelliJ IDEA y cree un nuevo proyecto Java titulado `TallerPOO_NombreApellido` .
- **Organización de Paquetes:** Dentro de la carpeta `src` , cree la estructura de paquetes `com.taller.poo` donde alojará la totalidad de las clases desarrolladas.
- **Trabajo Autónomo y Sin Soluciones:** Escriba y compile el código por su cuenta. Este documento establece requerimientos y casos de depuración sin proveer el código resuelto, impulsando el razonamiento lógico y la capacidad analítica.
- **Estilo de Código Standard:** Aplique de forma estricta las convenciones de nomenclatura Java (`camelCase` para atributos y métodos, `PascalCase` para clases y constantes en mayúsculas sostenidas).
- **Entorno de Ejecución:** Utilice exclusivamente las clases del Java Standard Development Kit (Java SDK estándar).

2. Sección 1: Modelado de Clases y Atributos

1.1. Ejercicio de Abstracción y Atributos: Clase `Vehiculo`

Diseñe la estructura base de una clase llamada `Vehiculo` aplicando los principios de abstracción de POO.

1. Cree el archivo `Vehiculo.java` dentro del paquete `com.taller.poo` .
2. Declare los siguientes atributos de instancia especificando un nivel de acceso privado (`private`):
   - Marca del vehículo (Tipo `String`).

- Modelo del vehículo (Tipo `String`).
- Kilometraje actual (Seleccione entre el primitivo `double` o el Wrapper `Double`, justificando conceptualmente su elección en un comentario de código).
- Velocidad máxima alcanzable (Seleccione entre el primitivo `int` o el Wrapper `Integer`, justificando conceptualmente su elección).
- Estado de encendido del motor (Seleccione entre el primitivo `boolean` o el Wrapper `Boolean`, justificando conceptualmente su elección).

3. Redacte un comentario de bloque (`/* ... */`) en la parte superior de la clase explicando el impacto en el consumo de memoria y la posibilidad de admitir valores nulos (`null`) al elegir Wrappers en lugar de tipos primitivos.

1.2. Ejercicio de Modelado de Entidades: Clase `Estudiante`

Especifique técnicamente la entidad `Estudiante` para un sistema de registro académico.

- **Nombre de la clase:** `Estudiante`
- **Paquete:** `com.taller.poo`
- **Atributos requeridos:**
  - Identificación o matrícula del estudiante (`identificacion` : `String`).
  - Nombre completo del estudiante (`nombreCompleto` : `String`).
  - Promedio general de calificaciones (`promedioCalificaciones` : primitivo `double`).
  - Estado de la matrícula o activo (`estaMatriculado` : primitivo `boolean`).
- **Requisito de nomenclatura y estado:** Aplique la convención estándar `camelCase` en la declaración de todas las variables miembros. Los atributos deben declararse privados (`private`) y no deben inicializarse directamente en sus líneas de declaración; la asignación del estado inicial debe delegarse exclusivamente a los constructores.

1.3. Ejercicio de Dominio Financiero: Clase `CuentaBancaria`

Cree la estructura de la clase `CuentaBancaria` integrando la gestión de tipos de datos financieros y comentarios explicativos en código.

Requerimientos:

1. Crear la clase `CuentaBancaria` en el paquete `com.taller.poo` con los siguientes atributos privados:
   - `numeroCuenta` (Tipo `String`).
   - `titular` (Tipo `String`).
   - `saldo` (Tipo primitivo `double`).
   - `limiteCredito` (Tipo Wrapper `Double`).
2. Incluya la siguiente plantilla de código en su proyecto y complete las respuestas analíticas dentro del bloque de comentario:

```java
package com.taller.poo;

public class CuentaBancaria {
    private String numeroCuenta;
    private String titular;
    private double saldo;
    private Double limiteCredito;

    /*
     * PREGUNTAS DE ANÁLISIS:
     * 1. ¿Qué diferencia ocurre a nivel de memoria y asignación por defecto al declarar 'saldo' como double primitivo frente a 'limiteCredito' como Wrapper Double?
     * 2. ¿Por qué una operación aritmética sobre un atributo no inicializado de tipo Double provoca una NullPointerException, mientras que sobre un double primitivo opera con 0.0?
     */
}
```

3. Sección 2: Métodos de Instancia y Métodos Estáticos (`static`)

2.1. Métodos de Instancia y Manipulación de Cadenas

Incorpore métodos de instancia en las clases definidas para procesar datos e implementar la lógica de negocio.

Clase	Método de Instancia	Requisito de Lógica Interna
`Estudiante`	`obtenerReporteAcademicos()`	Debe construir y retornar un `String` utilizand la clase `StringBuilder` en combinación con `String.format()` o `StringBuilder.append(String.format(...)` Utilice los especificadores `%s` , `%.2f` y `%b` par formatear el nombre, el promedio a dos decimales y el estado de la matrícula. **Nota de diseño:** Este método no debe imprimir directamente en consola mediante `System.out.printf` ; la impresión en pantalla es responsabilidad exclusiva de las clases ejecutables.
`CuentaBancaria`	`depositar(double monto)`	Debe validar mediante una estructura `if-else` que el monto recibido sea estrictamente mayo a 0. Si la validación es exitosa, incrementa el

		saldo y retorna `true`; de lo contrario, no altera el saldo y retorna `false`.
`CuentaBancaria`	`retirar(double monto)`	Debe validar mediante condicionales `if-else` que el monto sea positivo y que el saldo disponible (sumado al `limiteCredito` si este no es nulo) sea suficiente. Si cumple las condiciones, descuenta el monto del saldo y retorna `true`; si no, retorna `false`.

2.2. Métodos Estáticos Utilitarios (`static`)

Diseñe una clase utilitaria llamada `ConversorMoneda` que proporcione operaciones financieras rápidas sin depender de estados de instancia.

Nota Teórica: Los métodos estáticos pertenecen a la clase en sí y no a una instancia específica del objeto. Por tanto, pueden ser ejecutados directamente invocando el nombre de la clase sin instanciar objetos previamente mediante el operador `new`.

1. Cree la clase `ConversorMoneda` dentro del paquete `com.taller.poo`.
2. **Encapsulamiento de Clase Utilitaria:** Defina un constructor por defecto explícito con acceso privado (`private ConversorMoneda() {}`) para impedir que la clase utilitaria pueda ser instanciada accidentalmente desde cualquier otra parte del sistema.
3. Definir el método estático `convertirDolaresAEuros(double montoDolares, double tasaCambio)`:
   - **Entrada:** Monto en dólares (`double`), Tasa de cambio (`double`).
   - **Validación:** Si el monto o la tasa de cambio son menores o iguales a cero, retornar `0.0`.
   - **Salida:** Resultado de multiplicar el monto por la tasa de cambio (`double`).
4. Definir el método estático `formatearMoneda(double monto)`:
   - **Entrada:** Valor numérico (`double`).
   - **Salida:** Cadena de texto formateada retornado por `String.format("$%.2f", monto)`.

4. Sección 3: Constructores y Uso de `this`

3.1. Sobrecarga de Constructores e Inicialización

Implemente el concepto de sobrecarga de constructores (*constructor overloading*) en las clases `Vehiculo` y `CuentaBancaria` para permitir múltiples mecanismos de instanciación.

- **Requisitos para `**Vehiculo**`:**
  - **Constructor por defecto:** No recibe parámetros. Debe asignar los siguientes valores explícitos: marca `"Genérico"`, modelo `"Sin Modelo"`, kilometraje `0.0`, velocidad máxima `120`, estado encendido `false`.

    - **Constructor parametrizado:** Recibe argumentos para inicializar la totalidad de los atributos (`marca` , `modelo` , `kilometraje` , `velocidadMaxima` , `encendido`).
- **Requisitos para `**CuentaBancaria**` :**
    - **Constructor por defecto:** Asigna número de cuenta `"000000"` , titular `"Sin Nombre"` , saldo `0.0` y límite de crédito `0.0` .
    - **Constructor de 2 parámetros:** Recibe `numeroCuenta` y `titular` , inicializando el saldo en `0.0` y el límite de crédito en `100.0` .
    - **Constructor completo de 4 parámetros:** Recibe e inicializa la totalidad de los atributos de la clase (`numeroCuenta` , `titular` , `saldo` , `limiteCredito`).

3.2. Desambiguación de Parámetros con la Palabra Clave `this`

Al definir los constructores parametrizados, aplique el patrón donde los parámetros de entrada tengan exactamente el mismo nombre que los atributos de la clase.

- Utilice la palabra clave `this` para desambiguar el atributo de la instancia con respecto al argumento recibido en el método (ejemplo: `this.titular = titular;`).
- Verifique que ningún atributo permanezca con su valor por defecto debido a un error de sombreado de variables (*variable shadowing*).

5. Sección 4: Instanciación de Objetos y Manipulación del Estado

4.1. Creación e Interacción con Múltiples Objetos en la Clase `Principal`

Cree una clase ejecutable para verificar el comportamiento de las clases e interacción entre los diferentes objetos.

Lista de Cotejo para la Clase `**Principal**` :

- [ ] Crear la clase `Principal` en el paquete `com.taller.poo` que contenga el método `public static void main(String[] args)` .
- [ ] Instanciar al menos **3 objetos** de la clase `Vehiculo` mediante el operador `new` (uno con el constructor por defecto y dos con el constructor parametrizado).
- [ ] Instanciar al menos **3 objetos** de la clase `Estudiante` especificando datos representativos.
- [ ] Instanciar al menos **3 objetos** de la clase `CuentaBancaria` utilizando los tres constructores sobrecargados creados.
- [ ] Ejecutar operaciones de modificación de estado: Invocar los métodos `depositar()` y `retirar()` en las distintas cuentas bancarias.
- [ ] Invocar métodos estáticos de `ConversorMoneda` directamente desde la clase.

4.2. Alteración del Estado y Reporte Formateado

Imprima el estado final de los objetos en la consola desde la clase `Principal` utilizando únicamente salidas formateadas.

Especificaciones técnicas:

- Utilizar exclusivamente `System.out.printf` para enviar los datos formateados a la consola.
- Integrar los especificadores `%s` (cadenas), `%d` (enteros), `%.2f` (decimales a dos posiciones), `%b` (booleanos) y `%n` (salto de línea de plataforma).

Ejemplo de Salida Esperada en Consola:

```
=== REPORTE DE VEHÍCULOS ===
[1] Marca: Toyota | Modelo: Corolla | KM: 15000.50 | VelMax: 180 km/h | Encendido: true
[2] Marca: Genérico | Modelo: Sin Modelo | KM: 0.00 | VelMax: 120 km/h | Encendido: false

=== REPORTE DE CUENTAS BANCARIAS ===
Cuenta: 1001 | Titular: Carlos Pérez | Saldo Final: $1250.75 | Crédito: $500.00
Cuenta: 1002 | Titular: Ana Gómez | Saldo Final: $50.00 | Crédito: $100.00
```

6. Sección 5: Reto Integrador: Sistema de Gestión de Objetos en Consola

5.1. Definición del Dominio e Integración General

Desarrolle un programa interactivo ejecutado por consola denominado `SistemaGestionCuentas`. El programa gestionará un conjunto finito de cuentas bancarias utilizando un arreglo de objetos `CuentaBancaria[]`, integrando estructuras de control e interacción continua por teclado.

5.2. Requisitos Técnicos Obligatorios del Reto

1. **Captura de Entrada por Consola:** Instanciar la clase `java.util.Scanner` para capturar las opciones e insumos ingresados por el usuario.
2. **Estructura Menú Continuo:** Implementar un bucle `do-while` que mantenga activo el sistema hasta que el usuario elija explícitamente la opción de salida.
3. **Estructura de Menú Interactiva:** Implementar un `switch` para evaluar las siguientes opciones obligatorias del menú:
   - **Opción 1: Depositar en una cuenta.** Solicita el `numeroCuenta` y el monto a depositar. Busca la cuenta y ejecuta el método `depositar()`.
   - **Opción 2: Retirar de una cuenta.** Solicita el `numeroCuenta` y el monto a retirar. Busca la cuenta y ejecuta el método `retirar()`.

- **Opción 3: Ver reporte consolidado del sistema.** Recorre todas las cuentas, calcula el saldo total acumulado y muestra el reporte.
- **Opción 4: Salir.** Finaliza la ejecución del bucle `do-while`.

4. **Colección de Objetos:** Declarar e inicializar un arreglo de objetos `CuentaBancaria[]` de tamaño fijo (3 elementos) pre-poblado con cuentas instanciadas.
5. **Validaciones de Entrada (`**if-else**`):**
   - Validar que los montos ingresados por el usuario sean estrictamente mayores a cero.
   - Verificar la existencia de la cuenta mediante la coincidencia del `numeroCuenta`. En caso de no existir, presentar un mensaje de error formateado.
6. **Búsqueda e Iteración:** Implementar un bucle `for` o `while` para recorrer el arreglo al buscar una cuenta específica por su identificador `numeroCuenta` o al calcular totales.
7. **Construcción y Formato de Salida:** Utilizar un objeto `StringBuilder` dentro del reporte para acumular el detalle de las cuentas y desplegar la información final formateada con `System.out.printf`.

7. Sección 6: Análisis y Depuración de Código (Debugging POO en IntelliJ IDEA)

7.1. Casos de Estudio de Errores Comunes en POO

Analice los siguientes 4 fragmentos de código Java que contienen errores conceptuales o de sintaxis típicos en POO. Ingrese el código en su IDE IntelliJ IDEA, identifique la falla y responda por escrito las preguntas analíticas solicitadas para cada caso.

Caso de Estudio 1: Excepción de Referencia Nula (`NullPointerException`)

```
package com.taller.poo;

public class PruebaEstudiante {
    public static void main(String[] args) {
        Estudiante estudiante1;

        // Intento de invocación sobre una variable no instanciada
        String reporte = estudiante1.obtenerReporteAcademicos();
        System.out.println(reporte);
    }
}
```

Preguntas Analíticas:

1. ¿Qué error indica el compilador de Java o qué excepción se lanza en tiempo de ejecución al intentar compilar/ejecutar el código anterior?

2. ¿Por qué la variable `estudiante1` no puede invocar el método de instancia `obtenerReporteAcademicos()` en su estado actual?
3. Indique la instrucción exacta de código que debe agregarse para instanciar correctamente el objeto `estudiante1` mediante el operador `new` antes de invocar el método.

Caso de Estudio 2: Invocación de Contexto de Instancia desde Contexto Estático

```
package com.taller.poo;

public class GestorVehiculos {

    public void encenderMotor() {
        System.out.println("El motor del vehículo ha sido encendido.");
    }

    public static void main(String[] args) {
        // Intento de llamada directa a método de instancia desde el método
estático main
        encenderMotor();
    }
}
```

Preguntas Analíticas:

1. ¿Cuál es el mensaje de error emitido por el compilador de Java en la línea donde se invoca `encenderMotor()`?
2. Explique desde la perspectiva de la POO por qué un método con el modificador `static` (como `main`) no puede invocar directamente un método de instancia que requiere un estado de objeto.
3. Describa las dos alternativas de corrección válidas para que el código compile y ejecute correctamente el método.

Caso de Estudio 3: Firma Incorrecta en la Declaración de un Constructor

```
package com.taller.poo;

public class CuentaBancaria {
    private String numeroCuenta;
    private double saldo;

    // Declaración errónea del constructor
    public void CuentaBancaria(String numeroCuenta, double saldoInicial) {
        this.numeroCuenta = numeroCuenta;
```

```
        this.saldo = saldoInicial;
    }
}
```

Preguntas Analíticas:

1. ¿Por qué la inclusión de la palabra clave `void` altera la naturaleza del método y hace que Java deje de reconocerlo como un constructor?
2. Si desde otra clase se intenta ejecutar `CuentaBancaria cuenta = new CuentaBancaria("A-101", 500.0);`, ¿por qué la compilación reportará que no se encuentra un constructor adecuado?
3. Escriba la firma y declaración correcta del constructor omitiendo el tipo de retorno.

Caso de Estudio 4: Sombreado de Variables (*Variable Shadowing*) por Ausencia de `this`

```
package com.taller.poo;

public class Vehiculo {
    private int velocidadMaxima;
    private boolean encendido;

    public Vehiculo(int velocidadMaxima, boolean encendido) {
        // Asignación de variables omitiendo la referencia 'this'
        velocidadMaxima = velocidadMaxima;
        encendido = encendido;
    }

    public void imprimirEstado() {
        System.out.printf("Velocidad Máxima: %d | Encendido: %b%n",
velocidadMaxima, encendido);
    }
}
```

Preguntas Analíticas:

1. Si se instanciar la clase mediante `Vehiculo v = new Vehiculo(200, true);` y luego se invoca `v.imprimirEstado();`, ¿qué valores se proyectarán en la consola y por qué?
2. Explique el fenómeno de *sombreado de variables* (*variable shadowing*) y cómo afecta la asignación cuando el nombre del parámetro coincide con el del atributo.

3. Reescriba el bloque del constructor corrigiendo el error mediante la utilización de la palabra clave `this`.